Frontend Technical Round

Objective

Build a small CRUD application using **Next.js (App Router)** by following clean architecture, reusable components, and modern frontend best practices.

Submission Guidelines

- Create the project and push the completed code to your **own GitHub repository**.
- Share the GitHub repository link after completing the assignment.
- Provide repository access to **infiniqedeveloper@gmail.com**.
- Include a `README.md` with project setup and run instructions.
- Push your code **task-wise** with meaningful commit messages that clearly describe each completed task.
- When you **start coding**, send a message: **"Started Working"**.
- When you **finish the assignment**, send a message: **"Completed"**.
- For any assignment-related communication, contact: **+91 93137 36866**.

Project Setup

Initialize the project using **Next.js (App Router)** with **TypeScript** and **Tailwind CSS**, and configure the following brand colors throughout the application.

Role	Color	Hex
Primary	Royal Blue	#2563EB
Secondary	Slate Gray	#64748B
Accent	Emerald Green	#10B981

Tasks

Project Setup

1. Set up the project using **Next.js (App Router)** with **TypeScript**, **Tailwind CSS**, and configure **DM Sans** as the default application font.

2. Apply the provided **Primary**, **Secondary**, and **Accent** brand colors throughout the application.

3. Create a clean, scalable, and maintainable **module-wise folder structure**.

4. Follow consistent and proper **naming conventions** across the project (components, hooks, services, types, pages, utilities, etc.).

5. Configure the application **logo**, **favicon**, and **metadata** (title and description).

6. Create a custom **404 (Not Found)** page.

7. Configure **Axios** and **React Query (TanStack Query)** for API integration.

Product Module

8. Create a reusable **Table** component and use it in the **Product Listing** page.

- Use the **DummyJSON Products API**.
- API: https://dummyjson.com/products
- Display products in a table.

- Implement **server-side search**.
- Implement **server-side sorting**.
- Implement **server-side pagination**.

9. Create a **Product Details** page.

10. Create an **Add Product** page.

11. Create an **Edit Product** page.

12. Implement **Delete Product** functionality from the Product Listing page.

13. Preserve the current **search**, **sorting**, and **pagination** state when navigating to the **Product Details** or **Edit Product** page, and restore the same state when returning to the Product Listing page.

User Module

14. Implement **User CRUD** operations on a single page.

- Use the **DummyJSON Users API**.
- API: https://dummyjson.com/users
- Display users in a reusable table.
- Use a **Modal** for **Add** and **Edit** operations.
- Use a **Sidebar/Drawer** for **View User Details**.
- Implement **Delete** functionality from the listing.

Recipes Module

15. Create a **Recipes Listing** page.

- Use the **DummyJSON Recipes API**.
- API: https://dummyjson.com/recipes
- Display recipes in a responsive card layout.
- Implement search functionality.
- Implement pagination.

16. Create a **Recipe Details** page.

- Fetch recipe details using a **server-side API call**.
- API: `https://dummyjson.com/recipes/{id}`

Development Guidelines

- Use **Axios** for API integration.
- Use **React Query (TanStack Query)** for all client-side API calls.
- Use **React Hook Form** with **Yup** for form validation.
- Create reusable common components wherever applicable (e.g., Button, Input, Table, Modal, Drawer/Sidebar, Loader, Confirmation Dialog).
- Build a fully **responsive UI** that works seamlessly across **mobile**, **tablet**, **laptop**, and **desktop** screen sizes.
- Properly handle loading, success, and error states.
- Follow a clean and consistent folder structure.
- Write clean, reusable, maintainable, and type-safe code following frontend best practices.

Bonus

- Using **Shadcn UI** for reusable components is optional but recommended and will be considered a plus during the evaluation.

Timeline

- **Maximum Duration:** 6 Hours.
- Candidates are encouraged to complete the assignment within **5 hours**.
- The maximum allowed time for submission is **6 hours**.
- The assessment time will be calculated from the **"Started Working"** message until the **"Completed"** message.